EVPN 中文译文

> 原文：FRRouting 用户指南 · stable-10.1 · EVPN
> https://docs.frrouting.org/en/stable-10.1/evpn.html
> 整理日期：2026-10-20 | 简体中文译文（非官方）
> 正文完整翻译；配置命令、参数、示例输出及代码注释按原文保留。

EVPN 概念 FRR 配置 Linux 接口配置 查询命令

EVPN 是 Ethernet Virtual Private Network（以太网虚拟专用网络）的缩写。它是 BGP 的扩展，用于在公共网络上为桥接型（二层，L2）和路由型（三层，L3）VPN 提供信令。RFC 7432 对 EVPN 进行了定义，后续还有多份 RFC 和 IETF 草案对其进行了更新，包括 RFC 9135（以太网 VPN 中的集成路由与桥接）、RFC 9136（以太网 VPN 中的 IP 前缀通告）、RFC 8584（以太网 VPN 指定转发器选举的可扩展性框架）以及 RFC 8365（使用以太网 VPN 的网络虚拟化覆盖方案）。FRR 支持通过 LACP 以太网段，为多归属设备（MHD）提供全活（All-Active）二层多归属，并且同时支持对称和非对称 IRB（集成路由与桥接）。FRR 使用“基于 VLAN 的业务接口”（RFC 7432）实现 MAC-VRF，并按照“无接口的 IP-VRF 到 IP-VRF 模型”（RFC 9136）处理对称 IRB 路由。

1 EVPN 概念

BGP-EVPN 是以太网帧传输的控制平面，无论这些帧通过桥接还是路由方式转发。当基于 VLAN 的业务接口采用 VXLAN 封装时，使用单个 VNI 表示一个 EVPN 实例（EVI）；该实例拥有自己的路由区分符（Route Distinguisher，RD），以及一组用于导入和导出的路由目标（Route Target，RT）。

VNI 分为二层 VNI（关联 MAC-VRF）和三层 VNI（关联 IP-VRF），这一区别表明 VRF 所表示的信息类型。IP-VRF 表示路由表，其工作方式与传统 L3VPN 中的 VRF 基本相同；MAC-VRF 则表示桥接表，即 MAC（FDB，转发数据库）表项和 ARP/NDP 表项。

MAC-VRF 可以理解为一个 VLAN，它可以关联 SVI，也可以不关联。SVI 是绑定到桥接域的三层接口。在 Linux 中，SVI 可以是传统网桥，也可以是 VLAN 感知网桥上的 VLAN 子接口。如果 VLAN 存在 SVI，就可以将 ARP/NDP 表项与该广播域内的 MAC 地址绑定。如果没有 SVI，VLAN 则按传统二层方式工作，其中唯一可识别的主机地址类型是 MAC 地址。

多个 SVI 可以对应同一个 VRF；同样，多个 MAC-VRF（L2VNI）也可以对应同一个 IP-VRF（L3VNI）。在 FRR 中，L2VNI 关联哪个 L3VNI，取决于该 VLAN 是否存在 SVI，以及该 SVI 属于哪个 VRF。如果 L2VNI 没有 SVI，或者其 SVI 未加入任何 VRF，则该 L2VNI 会关联到 `default` VRF。如果 L2VNI 的 SVI 的主设备（master）是某个 VRF，则该 L2VNI 会关联到这个 VRF。

2 FRR 配置

FRR 通过 Netlink 从内核获知系统的 Linux 网络接口配置，但不会直接管理网络接口。下文给出了与 FRR 的 EVPN 实现兼容的 Linux 接口配置示例。虽然有多种接口管理工具可以正确配置内核（例如 ifupdown2），但这些示例使用 iproute2 添加和配置接口。

所有示例都采用相同的基本设置，但使用不同且彼此兼容的接口配置。

本示例将配置以下对象：

- 名为 `vrf1` 的 IP-VRF，关联 L3VNI 100。
- 名为 `vrf2` 的 IP-VRF，关联 L3VNI 200。
- 名为 `vrf3` 的 IP-VRF，不关联任何 L3VNI。
- 使用 VLAN 10 的 MAC-VRF，关联 L2VNI 110 和 IP-VRF `vrf1`。
- 使用 VLAN 20 的 MAC-VRF，关联 L2VNI 220 和 IP-VRF `vrf2`。
- 使用 VLAN 30 的 MAC-VRF，关联 L2VNI 330 和 IP-VRF `vrf3`。
- 使用 VLAN 40 的 MAC-VRF，关联 L2VNI 440 和 IP-VRF `default`。
- 使用 VLAN 50 的 MAC-VRF，关联 L2VNI 550，仅执行二层转发（L2-Only）。

2.1 配置示例

下面的 FRR 配置示例实现了上述 EVPN 环境。首先给出完整配置，随后逐项解释各个配置要素。

只要相应的 Linux 接口配置正确、兼容并且已经启用，以下配置就能建立正常工作的 EVPN 控制平面：

```
vrf vrf1
 vni 100
exit-vrf
!
vrf vrf2
 vni 200
exit-vrf
!
router bgp 4200000000
 neighbor 192.168.122.12 remote-as internal
 !
 address-family ipv4 unicast
  network 100.64.0.1/32
 exit-address-family
 !
 address-family l2vpn evpn
  neighbor 192.168.122.12 activate
  advertise-all-vni
  advertise-svi-ip
 exit-address-family
exit
!
router bgp 4200000000 vrf vrf1
 !
 address-family ipv4 unicast
  redistribute static
 exit-address-family
 !
 address-family ipv6 unicast
  redistribute static
 exit-address-family
 !
 address-family l2vpn evpn
  advertise ipv4 unicast
  advertise ipv6 unicast
 exit-address-family
exit
!
router bgp 4200000000 vrf vrf2
 !
 address-family ipv4 unicast
  redistribute static
 exit-address-family
 !
 address-family ipv6 unicast
  redistribute static
 exit-address-family
 !
 address-family l2vpn evpn
  advertise ipv4 unicast
  advertise ipv6 unicast
 exit-address-family
exit
```

在 `vrf` 配置段下执行 `vni` 命令，即可将 VRF 与 L3VNI 关联。在建立此关联之前，zebra 虽然会通过 Netlink 发现该 VNI，但会将其视为 L2VNI。可通过 `show evpn vni` 的输出查看某个 VNI 当前被视为二层还是三层 VNI。

以下配置告诉 zebra：将 VXLAN-ID 100 作为 `vrf1` 的 L3VNI，将 VXLAN-ID 200 作为 `vrf2` 的 L3VNI。

```
vrf vrf1
 vni 100
exit-vrf
!
vrf vrf2
 vni 200
exit-vrf
```

为了使 VXLAN 解封装正常工作，EVPN 环境中的其他 VTEP 必须能够到达本地 VTEP-IP（100.64.0.1）。本示例通过 BGP 的 `network` 语句通告本地 VTEP-IP，也可以使用静态路由，或 IS-IS、OSPF 等其他路由协议实现可达性。

要为 BGP 实例启用 EVPN，必须使用 `advertise-all-vni` 命令。本示例使用默认 VRF 承载 `l2vpn evpn` 地址族，因此会为默认 VRF 启用 EVPN。

本示例计划与 192.168.122.12 交换 EVPN 路由，因此需要为该对等体激活 `l2vpn evpn` 地址族，以允许通告和接收 EVPN NLRI（网络层可达性信息）。

`advertise-svi-ip` 命令同样应配置在已启用 EVPN 的 BGP 实例中。该命令让 FRR 为本地 SVI 接口关联的所有 MAC/IP 地址对生成本机（“self”）Type-2 路由。

```
router bgp 4200000000
 neighbor 192.168.122.12 remote-as internal
 !
 address-family ipv4 unicast
  network 100.64.0.1/32
 exit-address-family
 !
 address-family l2vpn evpn
  neighbor 192.168.122.12 activate
  advertise-all-vni
  advertise-svi-ip
 exit-address-family
exit
```

只有在相应 VRF 的 BGP 实例中配置了对应的 `advertise <afi> unicast` 命令后，来自该 IP-VRF 的 IPv4 和 IPv6 BGP 前缀才会作为 Type-5 路由导出到 EVPN。BGP RIB 中的所有路由，无论是本地产生、从对等体学习，还是从其他 VRF 泄漏而来，只要有效且在该 VRF 的单播表中被选中，就具备导出到 EVPN 的资格。

本示例中，`vrf1` 和 `vrf2` 的 BGP 实例使用 `redistribute static` 语句，将静态路由重分发到 `ipv4 unicast` 和 `ipv6 unicast` 地址族的 BGP 本地路由信息库（Loc-RIB）中。随后，`advertise ipv4 unicast` 和 `advertise ipv6 unicast` 命令会将这些单播前缀作为 Type-5 路由导出到 EVPN。

```
router bgp 4200000000 vrf vrf1
 !
 address-family ipv4 unicast
  redistribute static
 exit-address-family
 !
 address-family ipv6 unicast
  redistribute static
 exit-address-family
 !
 address-family l2vpn evpn
  advertise ipv4 unicast
  advertise ipv6 unicast
 exit-address-family
exit
!
router bgp 4200000000 vrf vrf2
 !
 address-family ipv4 unicast
  redistribute static
 exit-address-family
 !
 address-family ipv6 unicast
  redistribute static
 exit-address-family
 !
 address-family l2vpn evpn
  advertise ipv4 unicast
  advertise ipv6 unicast
 exit-address-family
exit
```

3 Linux 接口配置

Linux 内核提供了多种网络设备（netdevice）配置方式，可用于构建 EVPN-VXLAN 环境。以下给出与 FRR 兼容的网络设备配置示例，用于实现前文描述的环境。

配置时需要遵循以下基本原则：

- 本地 VTEP-IP 应始终使用 `lo` 设备上的一个可达 IP 地址。
- L3VNI 应始终具有一个 SVI，也称为 L3-SVI。
- 不应为 L3-SVI 配置任何 IP 地址，包括链路本地地址。
  - 可以通过 `addrgenmode none` 禁用 IPv6 地址自动配置。
- 仅当需要路由（IRB）或 ARP/ND 抑制时，L2VNI 才需要 SVI。
  - ARP/ND 抑制是内核功能，不由 FRR 管理。
  - 通过 `neigh_suppress` 为每个网桥从属端口（`bridge_slave`）启用 ARP/ND 抑制。
  - 仅应在 `vxlan` 接口上启用 ARP/ND 抑制。
  - 对于不用于路由（IRB）的 SVI，应禁用 IPv4/IPv6 转发。
- 用于 EVPN 的 VXLAN 接口应禁用动态 MAC/VTEP 学习。
  - 动态 MAC 学习是内核网桥驱动的功能，不是 FRR 的功能。
  - 通过 `learning {on|off}` 为每个 `bridge_slave` 开启或关闭动态 MAC 学习。
  - 动态 VTEP 学习是内核 VXLAN 驱动的功能，不是 FRR 的功能。
  - 通过 `[no]learning` 为每个 VXLAN 接口开启或关闭动态 VTEP 学习。
- VXLAN 接口不应配置 `remote` VTEP。
  - 远端 VTEP 通过 EVPN 学习，因此无需配置静态 VTEP。

3.1 传统网桥与传统 VXLAN 设备

在传统网桥模型中，每个 MAC-VRF 都使用一个独立的 `bridge` 接口，作为相应广播域的 SVI。当 `vlan_filtering` 设置为 `0`（禁用）时，网桥被视为“传统”网桥，表示该网桥只有一个广播域，且不依据 VLAN 标签区分广播域。同样，每个“传统” `vxlan` 接口仅承载一个 VNI。因此，在这种部署模型中，每个启用 VXLAN 的广播域，都包含一个加入传统网桥的传统 VXLAN 接口。

为 L3VNI 广播域创建的网桥应只有一个成员，即该 L3VNI 的 VXLAN 设备。为 L2VNI 广播域创建的网桥通常具有多个成员：该 L2VNI 的 VXLAN 设备，以及需要承载该二层域的主机端口或网络端口。

如果需要通过同一个主机端口或网络端口承载多个传统网桥的广播域，必须为每个广播域在该端口上创建一个带标签的 `vlan` 子接口，再将各 VLAN 子接口加入对应的传统网桥。这样可确保只有携带预期 VID 的报文才会归入相应的广播域。

```
###################
## vxlan vtep-ip ##
###################
ip addr add 100.64.0.1/32 dev lo
```

```
##############################
## ip-vrf vrf1 / l3vni 100 ##
##############################
ip link add vrf1 type vrf table 1100
ip link set vrf1 up
ip link add br100 type bridge
ip link set br100 master vrf1 addrgenmode none
ip link set br100 addr aa:bb:cc:00:00:64
ip link add vni100 type vxlan local 100.64.0.1 dstport 4789 id 100 nolearning
ip link set vni100 master br100 addrgenmode none
ip link set vni100 type bridge_slave neigh_suppress on learning off
ip link set vni100 up
ip link set br100 up
```

```
##############################
## ip-vrf vrf2 / l3vni 200 ##
##############################
ip link add vrf2 type vrf table 1200
ip link set vrf2 up
ip link add br200 type bridge
ip link set br200 master vrf2 addrgenmode none
ip link set br200 addr aa:bb:cc:00:00:c8
ip link add vni200 type vxlan local 100.64.0.1 dstport 4789 id 200 nolearning
ip link set vni200 master br200 addrgenmode none
ip link set vni200 type bridge_slave neigh_suppress on learning off
ip link set vni200 up
ip link set br200 up
```

```
##################
## ip-vrf vrf3 ##
##################
ip link add vrf3 type vrf table 1300
ip link set vrf3 up
```

```
################
## l2vni 110 ##
################
ip link add br10 type bridge
ip link set br10 master vrf1
ip link set br10 addr aa:bb:cc:00:00:6e
ip addr add 10.0.10.1/24 dev br10
ip addr add 2001:db8:0:10::1/64 dev br10
ip link add vni110 type vxlan local 100.64.0.1 dstport 4789 id 110 nolearning
ip link set vni110 master br10 addrgenmode none
ip link set vni110 type bridge_slave neigh_suppress on learning off
ip link set vni110 up
ip link set br10 up
```

```
###############
## l2vni 220 ##
###############
ip link add br20 type bridge
ip link set br20 master vrf2
ip link set br20 addr aa:bb:cc:00:00:dc
ip addr add 10.0.20.1/24 dev br20
ip addr add 2001:db8:0:20::1/64 dev br20
ip link add vni220 type vxlan local 100.64.0.1 dstport 4789 id 220 nolearning
ip link set vni220 master br20 addrgenmode none
ip link set vni220 type bridge_slave neigh_suppress on learning off
ip link set vni220 up
ip link set br20 up
```

```
###############
## l2vni 330 ##
###############
ip link add br30 type bridge
ip link set br30 master vrf3
ip link set br30 addr aa:bb:cc:00:01:4a
ip addr add 10.0.30.1/24 dev br30
ip addr add 2001:db8:0:30::1/64 dev br30
ip link add vni330 type vxlan local 100.64.0.1 dstport 4789 id 330 nolearning
ip link set vni330 master br30 addrgenmode none
ip link set vni330 type bridge_slave neigh_suppress on learning off
ip link set vni330 up
ip link set br30 up
```

```
###############
## l2vni 440 ##
###############
ip link add br40 type bridge
ip link set br40 addr aa:bb:cc:00:01:b8
ip addr add 10.0.40.1/24 dev br40
ip addr add 2001:db8:0:40::1/64 dev br40
ip link add vni440 type vxlan local 100.64.0.1 dstport 4789 id 440 nolearning
ip link set vni440 master br40 addrgenmode none
ip link set vni440 type bridge_slave neigh_suppress on learning off
ip link set vni440 up
ip link set br40 up
```

```
###############
## l2vni 550 ##
###############
ip link add br50 type bridge
ip link set br50 addrgenmode none
ip link set br50 addr aa:bb:cc:00:02:26
ip link add vni550 type vxlan local 100.64.0.1 dstport 4789 id 550 nolearning
ip link set vni550 master br50 addrgenmode none
ip link set vni550 type bridge_slave neigh_suppress on learning off
sysctl -w net.ipv4.conf.br50.forwarding=0
sysctl -w net.ipv6.conf.br50.forwarding=0
ip link set vni550 up
ip link set br50 up
```

```
##################
## create vlan subinterface of eth0 for each l2vni vlan and enslave each
## subinterface to the corresponding bridge
##################
ip link set eth0 up
for i in 10 20 30 40 50; do
   ip link add link eth0 name eth0.$i type vlan id $i;
   ip link set eth0.$i master br$i;
   ip link set eth0.$i up;
done
```

首先，创建名为 `vrf1` 的 `vrf` 接口，并将其绑定到 ID 为 1100 的内核路由表。该接口代表 IP-VRF `vrf1`，稍后将为其分配 L3VNI。

```
ip link add vrf1 type vrf table 1100
```

以下命令创建名为 `br100` 的传统 `bridge` 接口，将其绑定到 `vrf1`，禁用 IPv6 地址自动配置，并静态指定 `br100` 的 MAC 地址。该传统网桥用于映射到 `vrf1` 的 L3VNI 广播域，也就是说，`br100` 是 `vrf1` 的 L3-SVI。

```
ip link add br100 type bridge
ip link set br100 master vrf1 addrgenmode none
ip link set br100 addr aa:bb:cc:00:00:64
```

下面创建名为 `vni100` 的传统 `vxlan` 接口，使用 100.64.0.1 作为 VTEP-IP，承载 VNI 100，并禁用动态 VTEP 学习。随后禁用 `vni100` 的 IPv6 地址自动配置，将该接口加入 `br100`，启用 ARP/ND 抑制，并禁用动态 MAC 学习。

```
ip link add vni100 type vxlan local 100.64.0.1 dstport 4789 id 100 nolearning
ip link set vni100 master br100 addrgenmode none
ip link set vni100 type bridge_slave neigh_suppress on learning off
```

至此，VRF 和 L3VNI 的必要配置已完成。

接下来创建名为 `br10` 的传统网桥。通过将 `br10` 的 `master` 设置为 `vrf1`，把 `br10` 加入 `vrf1`。并非必须静态设置 SVI 的 MAC 地址，这里这样做是为了保持一致。由于 `br10` 将用于路由，因此还需为该 SVI 配置 IPv4 和 IPv6 地址。

```
ip link add br10 type bridge
ip link set br10 master vrf1
ip link set br10 addr aa:bb:cc:00:00:6e
ip addr add 10.0.10.1/24 dev br10
ip addr add 2001:db8:0:10::1/64 dev br10
```

如果 SVI 不用于路由，则不应为其配置 IP 地址，并且应通过对应的 sysctl 参数禁用该 SVI 的 IPv4/IPv6 `forwarding`。

```
sysctl -w net.ipv4.conf.<ifname>.forwarding=0
sysctl -w net.ipv6.conf.<ifname>.forwarding=0
```

以下命令为 VNI 100 创建 `vxlan` 接口。除 VNI 编号外，L2VNI 和 L3VNI 的接口设置相同。

> 译注：原文此处写作“VNI 100”，但下方命令为 `vni110`，且明确指定 `id 110`，因此此处应指 VNI 110；命令按原文保留。

```
ip link add vni110 type vxlan local 100.64.0.1 dstport 4789 id 110 nolearning
ip link set vni110 master br10 addrgenmode none
ip link set vni110 type bridge_slave neigh_suppress on learning off
```

最后，要将传统网桥的广播域限定为匹配特定 VLAN-ID 的流量，需要在主机端口或网络端口上创建 `vlan` 子接口。下例在 `eth0` 上创建名为 `eth0.10`、匹配 VID 10 的 VLAN 子接口。将 `eth0.10` 而非 `eth0` 加入 `br10`，可以确保只有从 `eth0` 进入且带有 VID 10 标签的以太网帧，才会归入 `br10` 的广播域。

```
ip link add link eth0 name eth0.10 type vlan id 10
ip link set eth0.10 master br10
```

如果不希望按 VLAN-ID 限定广播域，可以跳过 VLAN 子接口的创建，直接将 `eth0` 加入 `br10`。

```
ip link set eth0 master br10
```

至此，L2VNI 的必要配置已完成。

3.2 查看 EVPN 信息

```
show evpn mac vni (1-16777215) detail [json]
```

显示指定 VNI 的 MAC 地址详细信息。

```
show vrf [<NAME$vrf_name|all$vrf_all>] vni [json]
```

显示 VRF 到 L3VNI 的映射，同时显示该 L3VNI 关联的路由器 MAC 地址（router-mac）、SVI 接口和 VXLAN 接口。在 CLI 命令末尾添加 `json` 关键字，即可获取 JSON 格式的输出。

```
tor2# show vrf vni
VRF                                 VNI        VxLAN IF            L3-SVI              State Rmac
sym_1                               9288       vxlan21             vlan210_l3          Up    21:31:36:ff:ff:20
sym_2                               9289       vxlan21             vlan210_l3          Up    21:31:36:ff:ff:20
sym_3                               9290       vxlan21             vlan210_l3          Up    21:31:36:ff:ff:20
tor2# show vrf sym_1 vni
VRF                                 VNI        VxLAN IF            L3-SVI              State Rmac
sym_1                               9288       vxlan21             vlan210_l3          Up    44:38:36:ff:ff:20
```

版权与许可

本文件译自 FRRouting 10.1 用户指南的 EVPN 章节。原文版权归相应权利人所有。本译文作为衍生作品，依照原手册相同的许可条款发布；以下保留版权与许可声明原文。

原文页面版权标注：© Copyright 2017, FRR.
手册版权与许可来源：FRRouting 10.1 用户指南 — Copyright notice

Copyright (c) 1996-2018 Kunihiro Ishiguro, et al.

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of this manual under the conditions for verbatim copying, provided that the entire resulting derived work is distributed under the terms of a permission notice identical to this one.

Permission is granted to copy and distribute translations of this manual into another language, under the above conditions for modified versions, except that this permission notice may be stated in a translation approved by Kunihiro Ishiguro.